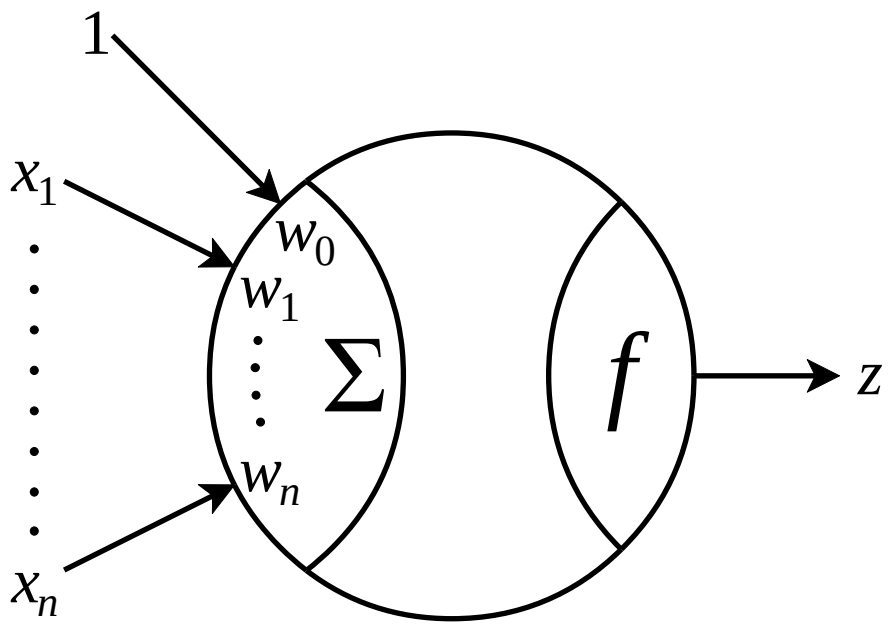
フィードフォワード・ネットワーク (FNN)と誤差逆伝播法

ニューロセルの基本構造（再掲）

現代的なモデル

$x_1 \sim x_n$　：入力（アナログ値もOK）
$w_1 \sim w_n$　：重み
$w_0\ (=-\theta)$：バイアス（$x_0 = 1$と考える）
f　：活性化関数
（シグモイド関数，ReLU等）

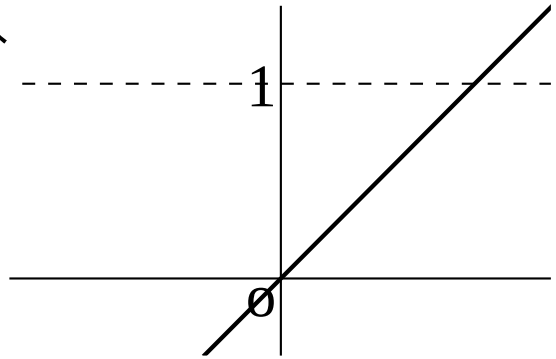


活性値：$u = \sum_{x=0}^{n} w_i x_i \quad (= w_0 + w_1 x_1 + \cdots + w_n x_n)$

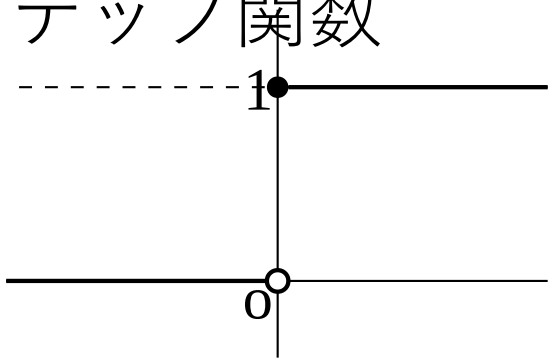
出力：$z = f(u)$　（アナログ値もOK）

活性化関数（再掲）

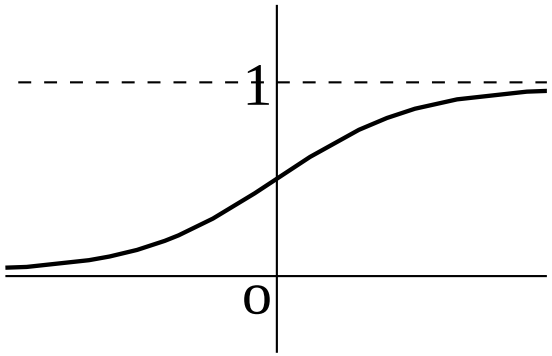
- 恒等関数



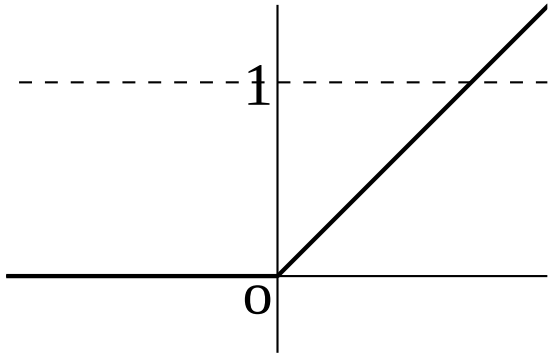
- ステップ関数



- シグモイド関数



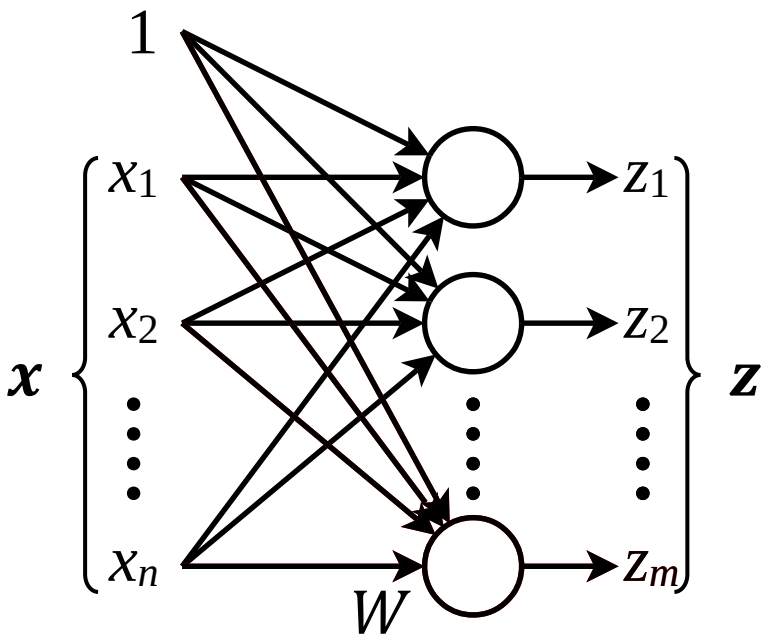
- 正規化線形関数 (ReRU)



活性化関数（再掲）

- 恒等関数 → 線形近似式，可微分
  - 多層化が無意味
- ステップ関数 → 線形判別式
- シグモイド関数：可微分
- 正規化線形関数 (ReRU)：片側可微分
- 他（さまざまに工夫されている）

n入力m出力単層ネットワーク（再掲）

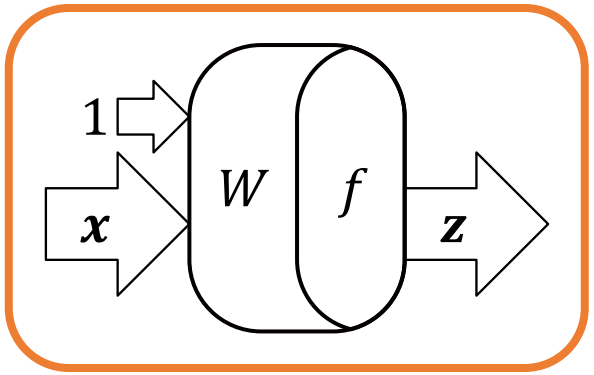


$$\begin{pmatrix} u_1 \\ \vdots \\ u_m \end{pmatrix} = \begin{pmatrix} w_{10} & w_{11} & \cdots & w_{1n} \\ \vdots & \vdots & \ddots & \vdots \\ w_{m0} & w_{m1} & \cdots & w_{mn} \end{pmatrix} \begin{pmatrix} 1 \\ x_1 \\ \vdots \\ x_n \end{pmatrix}$$

$$\boldsymbol{u} = \begin{pmatrix} \boldsymbol{w}_{*0} & W' \end{pmatrix} \begin{pmatrix} 1 \\ \boldsymbol{x} \end{pmatrix}$$

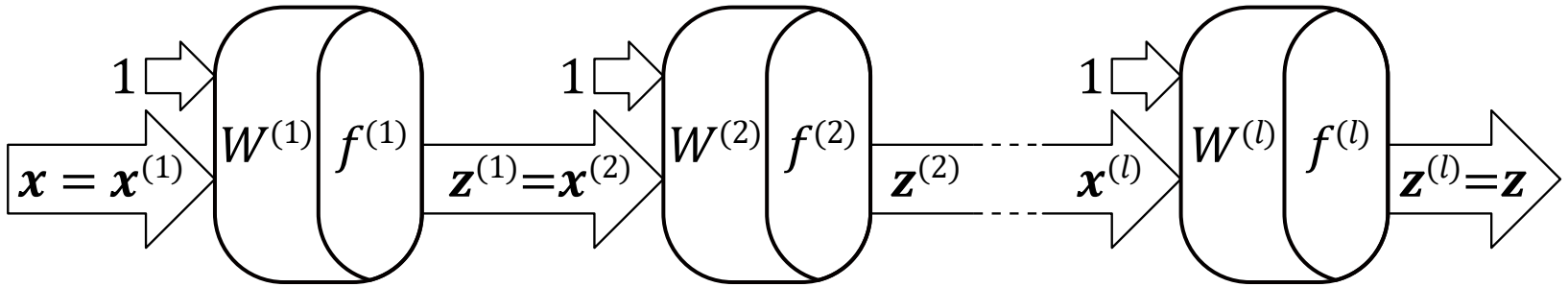
$$\boldsymbol{u} = W \begin{pmatrix} 1 \\ \boldsymbol{x} \end{pmatrix}$$

$$\boldsymbol{z} = f(\boldsymbol{u})$$



フィードフォワードネットワーク (FNN) （再掲）

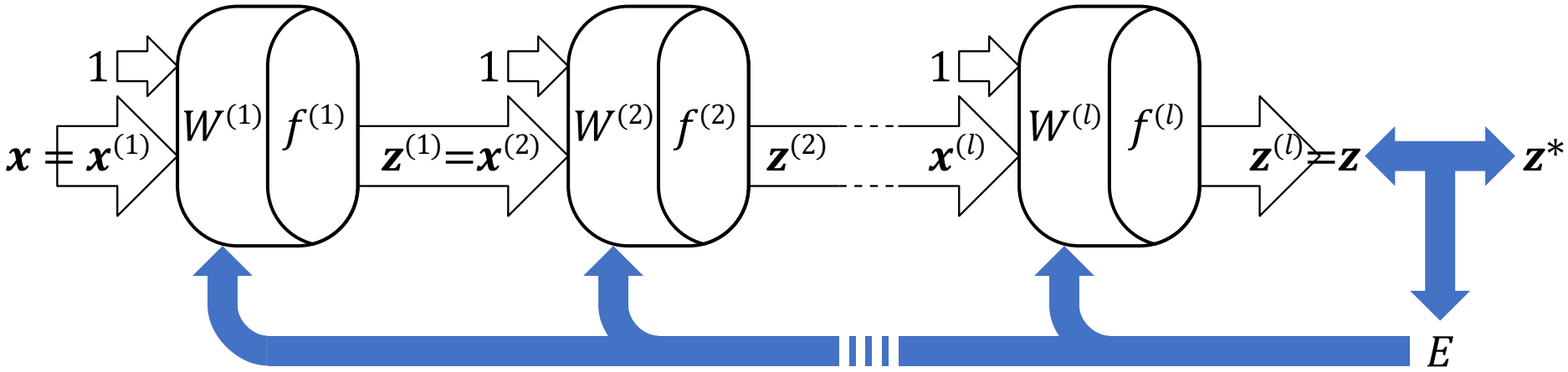
- 単層ネットワークを複数直列に並べる



※ 活性化関数 $f^{(k)}$ は，各層内では共通だが，層ごとに異なってもよい

FNNの学習

- 訓練例(x, z^*)の入力信号xに対する出力信号zと教師信号z^*の誤差Eを小さくするように，各層の$W^{(k)}$（重みとバイアス）を調整する



※ パーセプトロンと異なり，出力層（最終層）より前の層も調整する．

出力層の活性化関数と誤差

回帰タスクの場合

- 通常，出力データは実数値(の組)となるため，活性化関数には，例えば恒等関数等を用いる
- 誤差Eの計算には $\boldsymbol{z}^*$と$\boldsymbol{z}$ の二乗誤差（差の二乗和）や距離 (ユークリッド距離，ハミング距離) などを用いる

出力層の活性化関数と誤差

2クラスに分類するタスクの場合

- 一方のクラスに属する確率を出力する1つの出力ユニットを用意する（他方のクラスに属する確率は，その値を1から引いた値となる）
- 値を0〜1の範囲にするため，活性化関数にはシグモイド関数等が用いられる
- 誤差Eの計算には，（z^*とzの）交差エントロピー（下式）などを用いる

$$E = -z^* \log z - (1 - z^*) \log(1 - z) = \begin{cases} -\log z & (z^* = 1 \text{ のとき}) \\ -\log(1 - z) & (z^* = 0 \text{ のとき}) \end{cases}$$

出力層の活性化関数と誤差

3つ以上のクラスに分類するタスクの場合

- クラスごとにユニットを設け，そのクラスに属する確率を出力させると考える．活性化関数にはsoftmax関数等を用いる
- 誤差Eの計算には交差エントロピーを用いる

※ 詳細は割愛する

誤差逆伝播法（Back Propergation; BP）
（複層のFNNを効率よく学習させる手法）

- 考え方
  - 活性化関数（と誤差を計算する関数）を可微分なものにする
    - 誤差Eが，重みパラメータ$(w_{ji}^{(k)})$の可微分な関数になる
  - 誤差にE対する各$w_{ji}^{(k)}$ごとの偏微分係数(勾配) $\frac{\partial E}{\partial w_{ji}^{(k)}}$を計算
    - k についての漸化式を用いて計算される（$k+1$ の場合から k の場合を計算する … 「逆伝播」の由来）

    ※ 詳細な計算方法は割愛（別途参考資料を参照）

誤差逆伝播法（Back Propergation; BP）

- 考え方（続き）
  - 最急降下法(勾配降下法)を用いて各$w_{ji}^{(k)}$を修正する
    （誤差Eが小さくなるように，勾配最大の方向へ，
    勾配に比例した幅で修正する）
  - $w_{ji}^{(k)}$の修正値$\Delta w_{ji}^{(k)}$は以下のようになる

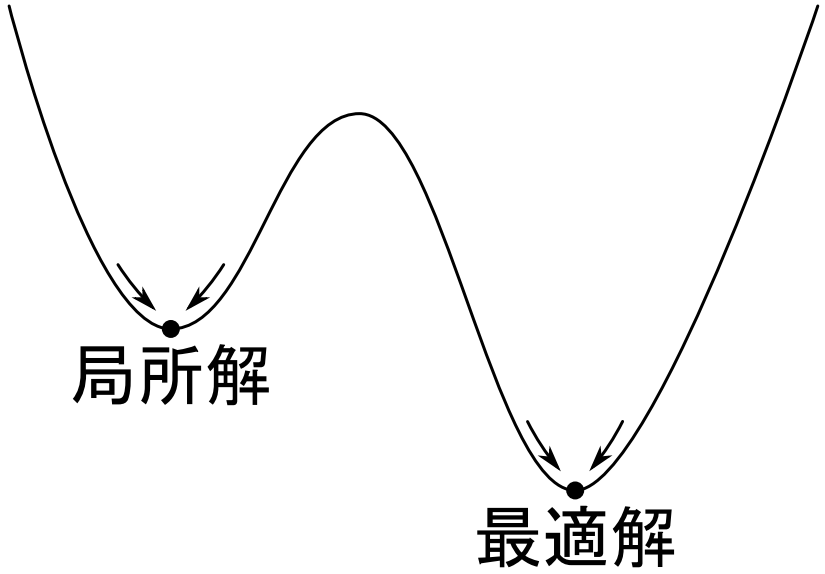
$$\Delta w_{ji}^{(k)} = -\lambda \frac{\partial E}{\partial w_{ji}^{(k)}} \qquad (\lambda：学習係数)$$

BPの問題点(1) ― 局所解問題

- 勾配を下っていって勾配が**0**になると，それ以上学習は進まないが，そこが最適解（誤差最小の解）であるとは限らない（極小点＝局所解 であるかもしれない）

〔対策〕

- 異なった初期状態で何度か実行するなど
- 多くの場合，局所解でも十分良い解が得られるという予想もある



BPの問題点(2) ― 勾配消失問題

- 偏微分係数が0に近いと，各重み成分の修正量$\Delta w_{ji}^{(k)}$が小さくなり，学習がなかなか進まない
- かといって，学習係数を大きくしすぎると，解を飛び越えてなかなか収束しなくなる可能性がある
- 出力層から遠い層（入力に近い層）ほど顕著に表れる
  （$w_{ji}^{(k)}$の出力に及ぼす影響が相対的に小さい）

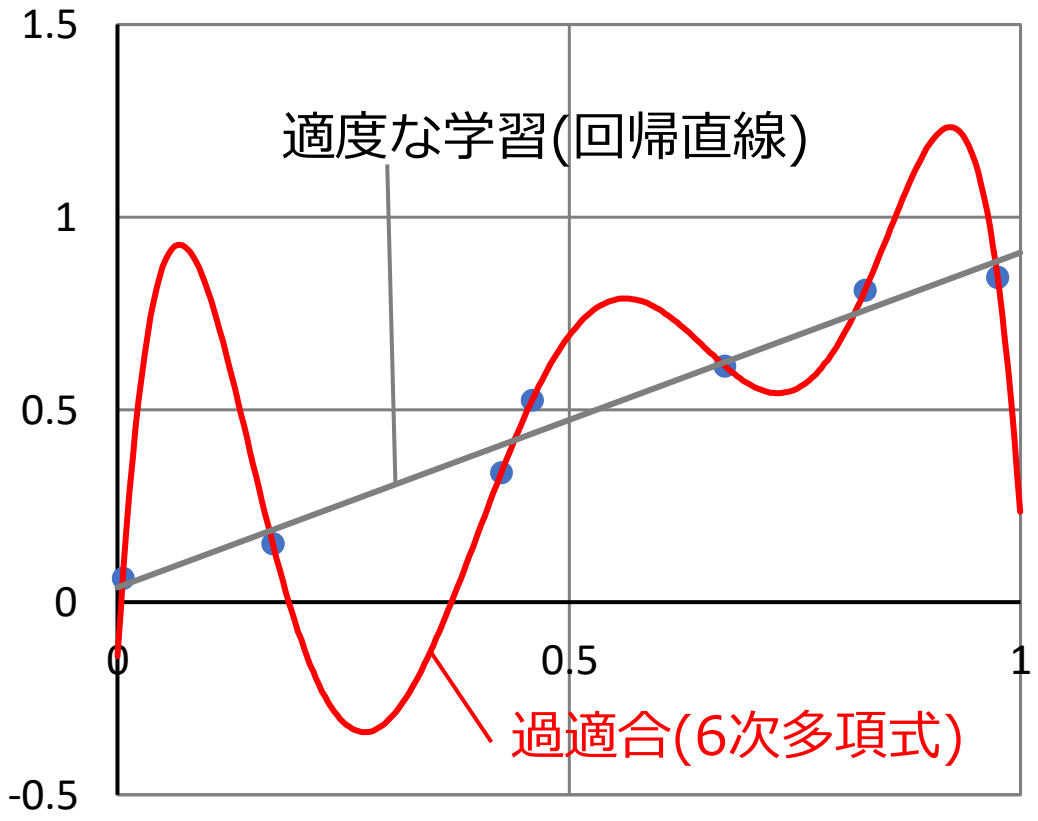
BPの問題点(2) — 勾配消失問題

〔対策〕

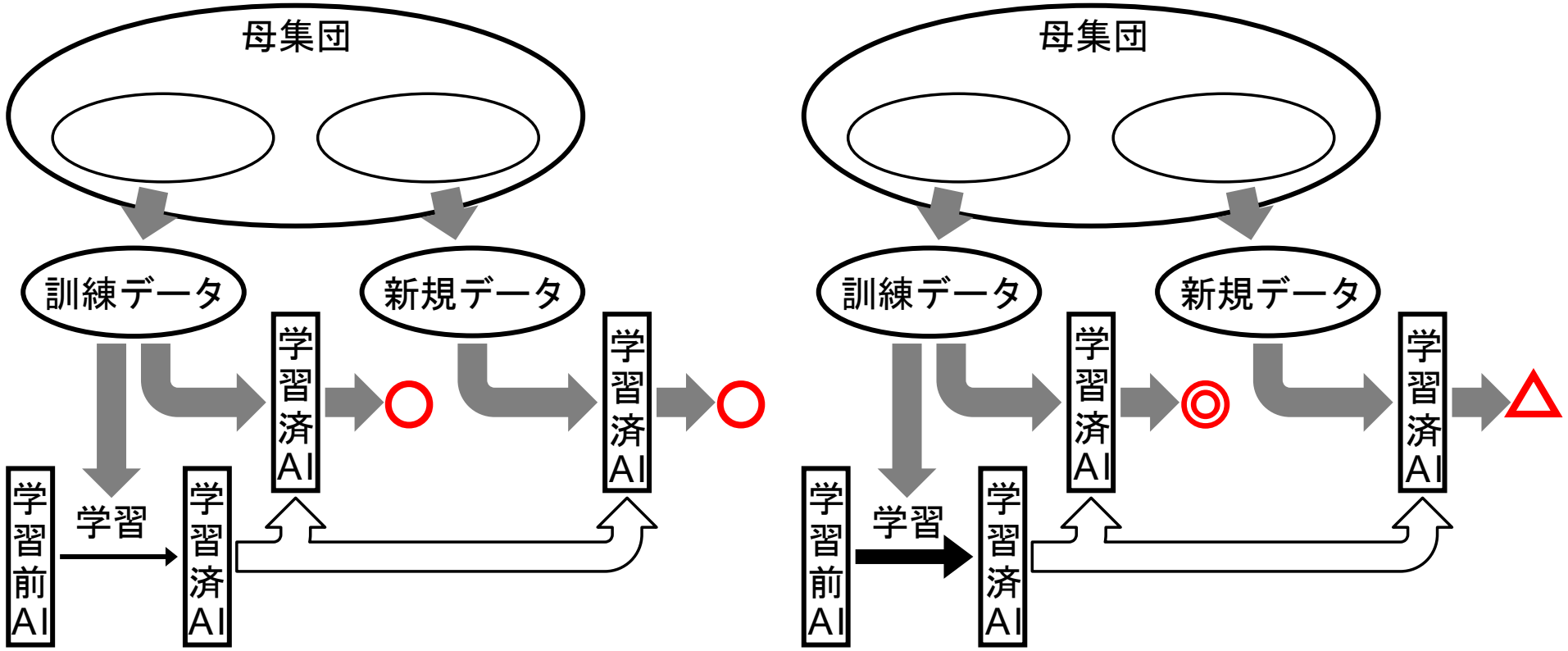
- 活性化関数の勾配と関係するので，活性化関数で工夫する．
(ReLU (正規化線形関数) 等を用いる)
- 出力層から遠い層には，事前に別の学習技法で学習させておく．
  - 自己符号化器による事前学習 (後述)
  - 転移学習 (他で学習したものを流用する)

BPの問題点(3) — 過適合
（オーバーフィッティング，過学習とも）

- 訓練データで学習したAIが，(未知の) 新規データに対しても適切な出力を出せるようになることを汎化という
- 訓練例に過度に最適化されて，かえって一般性を失う (新規データに対して適切な反応ができなくなる) 現象を過適合という



BPの問題点(3) ― 過適合
（オーバーフィッティング，過学習とも）

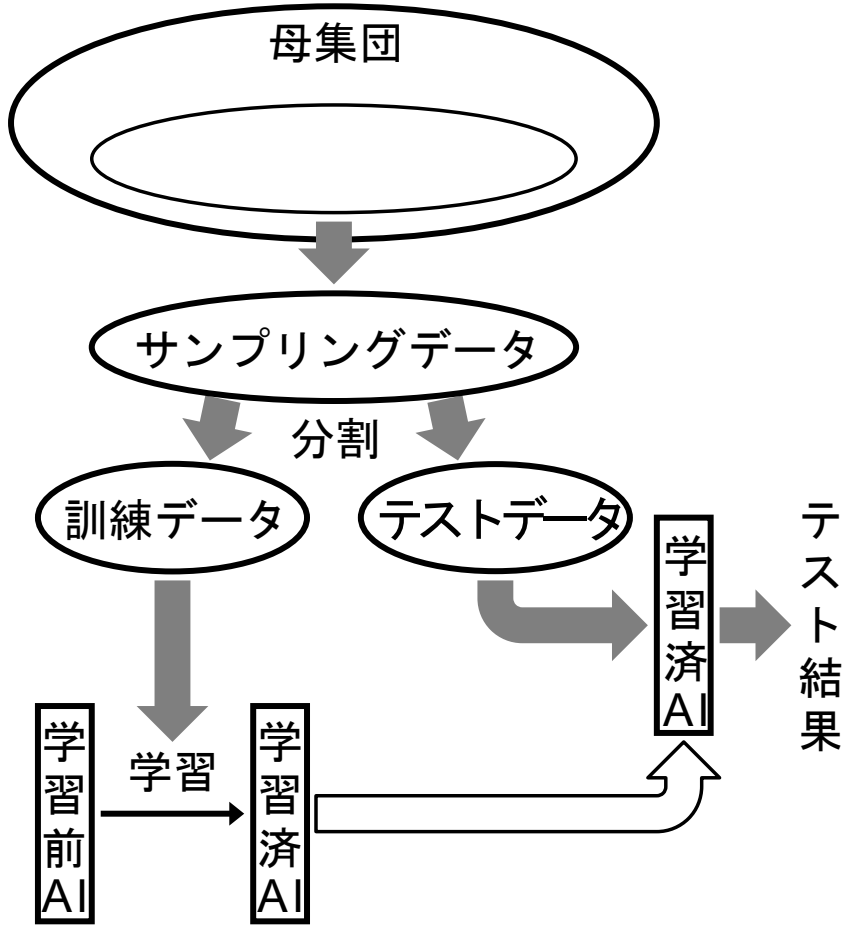


汎化がうまくいった場合　　過適合

BPの問題点(3) ― 過適合
（オーバーフィッティング，過学習とも）

〔対策〕

- 学習後に，テストデータを用いて過適合になっていないかチェックする
- 過適合になっていたら，学習をやり直す
  - 学習を停止するタイミングを見計らう
  - 学習モデルを修正する
- テストデータは，母集団から得たデータを訓練データと分けて用意する（右図）



BPの問題点(3) ― 過適合
（オーバーフィッティング，過学習とも）

〔対策；つづき〕

- パラメータを実質的に減らす（0 に近づける）よう圧力をかける．
  → 誤差関数に正則化項（ペナルティ項）を設ける
  - L1正則化（パラメタの絶対値の和）
  - L2正則化（パラメタの二乗和）
- ドロップアウト：学習時，各層でランダムに出力を0にする
  - 入力層：$p = 20\%$ 程度，隠れ層：$p = 50\%$ 程度（pは0にする確率）
  - 0にする出力は訓練データを一巡する間は固定して，反復ごとに変える
  - 推論時は 0 にせず，出力に $1 - p$ を掛けて調整する
  - 部分ネットワークの多数決をとる感じ